Ashwath Raj M

AP25110010712 | Core CSE A

LAB TASK – 11

Matrix Chain Multiplication using Dynamic Programming

Course	CSE-204 – Design and Analysis of Algorithms
Faculty	Mahesh Kumar Morampudi
Task Date	October 31
Points	10

Problem

Implement Matrix Chain Multiplication using Dynamic Programming to determine the minimum number of scalar multiplications required to multiply a sequence of matrices. Also display an optimal parenthesization.

Method

Let p[0...n] represent the matrix dimensions, where matrix Ai has dimensions p[i–1] × p[i]. Let dp[i][j] be the minimum multiplication cost for matrices Ai through Aj.

```
dp[i][j] = min { dp[i][k] + dp[k+1][j] + p[i-1] × p[k] × p[j] }
             i ≤ k < j
Base case: dp[i][i] = 0
```

The algorithm evaluates subchains in increasing order of length. A split table records the optimal split point and is used to reconstruct the parenthesization.

Sample Input

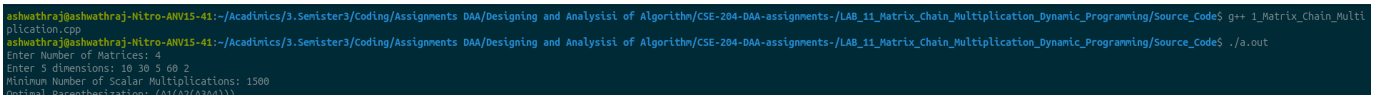
```
4
10 30 5 60 2
```

The four matrices have dimensions 10×30, 30×5, 5×60, and 60×2.

Output:

```
Enter Number of Matrices: 4
Enter 5 dimensions: 10 30 5 60 2
Minimum Number of Scalar Multiplications: 1500
Optimal Parenthesization: (A1(A2(A3A4)))
```

Program Output Screenshot



Code:

```
/*
Program Name: Matrix Chain Multiplication using Dynamic Programming
Program Description: Finds the minimum scalar multiplication cost and
                     optimal parenthesization of a matrix chain.
Program Creation Time: Oct 31, 2026
Program Creator: Ashwath Raj | AP25110010712
*/

#include <iostream>
#include <vector>
#include <limits>
using namespace std;

void printParenthesization(const vector<vector<int>>& split, int i, int j) {
    if (i == j) {
        cout << "A" << i;
        return;
    }
    cout << "(";
    printParenthesization(split, i, split[i][j]);
    printParenthesization(split, split[i][j] + 1, j);
    cout << ")";
}

int main() {
    int n;
    cout << "Enter Number of Matrices: ";
    cin >> n;

    if (!cin || n <= 0) {
        cerr << "Invalid number of matrices.\n";
        return 1;
    }

    vector<long long> p(n + 1);
    cout << "Enter " << n + 1 << " dimensions: ";
    for (long long& x : p) {
        cin >> x;
        if (!cin || x <= 0) {
            cerr << "Invalid dimensions. All dimensions must be positive.\n";
            return 1;
        }
    }

    const long long INF = numeric_limits<long long>::max();
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));
    vector<vector<int>> split(n + 1, vector<int>(n + 1, 0));

    for (int len = 2; len <= n; ++len) {
        for (int i = 1; i <= n - len + 1; ++i) {
            int j = i + len - 1;
            dp[i][j] = INF;

            for (int k = i; k < j; ++k) {
                __int128 cost = static_cast<__int128>(dp[i][k]) +
                    dp[k + 1][j] +
                    static_cast<__int128>(p[i - 1]) * p[k] * p[j];

                if (cost < dp[i][j] && cost <= INF) {
                    dp[i][j] = static_cast<long long>(cost);
                    split[i][j] = k;
                }
            }

            if (dp[i][j] == INF) {
                cerr << "Multiplication cost exceeds the supported range.\n";
                return 1;
            }
        }
    }

    cout << "Minimum Number of Scalar Multiplications: " << dp[1][n] << "\n";
    cout << "Optimal Parenthesization: ";
    printParenthesization(split, 1, n);
    cout << "\n";
    return 0;
}
```

Complexity

Time Complexity: $O(n^3)$, with $O(n^2)$ subproblems and up to $O(n)$ split positions per subproblem.
Space Complexity: $O(n^2)$, for the dynamic programming cost and split tables.